

Curso: Análisis Estadístico y Data Mining (ISIL 2026-1)

Alumno: Carlos Enrique Gil Carrillo

Contexto general

Una empresa peruana de servicios logísticos desea implementar un proyecto de análisis de datos para revisar entregas, reclamos y tiempos de atención. Sin embargo, enfrenta desafíos iniciales:

- Infraestructura limitada: colaboradores con computadoras de poca memoria y procesadores antiguos
- Falta de entorno: no cuentan con Python ni herramientas de análisis configuradas
- Objetivo: preparar el ambiente de forma ordenada para importar archivos Excel/CSV y generar reportes

Las preguntas evaluadas buscan validar si se entienden los requisitos, herramientas y librerías necesarias para levantar este proyecto correctamente.

Pregunta 01

Requisitos básicos de hardware y software antes de instalar Python

Según Clase 9, antes de instalar Python es crítico verificar que el equipamiento cumple con especificaciones mínimas. De lo contrario, scripts de análisis de datos se ejecutarán lentamente o fallarán.

A. Requisitos de Hardware

Componente	Mínimo Aceptable	Recomendado	Por qué importa en logística
CPU (Procesador)	Intel i5 / Ryzen 5	Intel i7 / Ryzen 7	Procesa cálculos de estadística y clustering con datasets grandes
Memoria RAM	8 GB	16 GB o más	Carga enteros datasets de entregas en memoria para análisis rápido
Disco	512 GB SSD	512 GB+ SSD	Almacena librerías Python, archivos CSV/Excel y resultados de análisis
GPU	No esencial	Recomendable	Solo si usan deep learning; para estadística básica no es necesaria

Justificación en el caso logístico:

- Si un colaborador intenta abrir un archivo de 50 MB con 5 años de entregas en pandas, con 4 GB RAM fallará.
- SSD garantiza que Jupyter Notebook o VS Code se inicie rápido.
- Procesador i5 mínimo permite que cálculos de correlación y regresión no tarden horas.

B. Requisitos de Software

1. Sistema Operativo compatible

- Windows 10 o superior, macOS, o Linux
- Verificar: Windows debe tener actualizaciones del sistema (OS Update)

2. Instalación base de Python

- Opción 1: Anaconda o Miniconda
 - Incluye Python 3.10+ pre-configurado
 - Gestor de paquetes Conda para instalar pandas, numpy, matplotlib
 - Jupyter Notebook integrado
 - Ventaja: idealmente para ciencia de datos sin conflictos de dependencias
- Opción 2: Python directo desde python.org
 - Versión 3.9 o superior
 - Requiere gestión manual de paquetes con `pip`

3. Gestor de paquetes

- Si usa Anaconda: `conda` (incluido)
- Si usa Python directo: `pip` (incluido en Python 3.9+)

4. IDE o editor

1. Jupyter Notebook (via Anaconda)
2. VS Code + extensión Python (recomendado para proyectos organizados)
3. RStudio si también usa R

5. Git (opcional pero recomendado)

- Control de versiones para código de análisis
- Facilita colaboración entre analistas

C. Verificación práctica en la empresa

Checklist a ejecutar en cada computadora:

1. Ver versión de Python

`python --version`

Esperado: Python 3.9 o superior

2. Verificar RAM disponible

En Windows: Task Manager > Rendimiento > Memoria

En Mac: Activity Monitor

En Linux: `free -h`

3. Verificar espacio en disco

En Windows: Propiedades C:\ > Espacio disponible

En Mac: Apple > Acerca de > Almacenamiento

En Linux: `df -h`

4. Si tiene Anaconda, verificar que conda funciona

`conda --version`

Si algún equipo no cumple mínimos, se debe gestionar actualización o considerar usar Google Colab como alternativa temporal en nube.

Pregunta 02

Para qué sirven pandas y NumPy en análisis de Excel/CSV

Según Clase 9 (Sección 4), pandas y NumPy son las dos librerías más críticas para manipular datos estructurados en Python. En el caso de logística, son indispensables.

A. NumPy: Operaciones numéricas y matrices

¿Qué es?

NumPy (Numerical Python) es una librería para operaciones matemáticas y manipulación de vectores/matrices. Sirve de base para casi todas las otras librerías de análisis.

¿Para qué sirve en logística?

Cálculos rápidos en arreglos

- Calcular promedio de tiempos de entrega: ``np.mean(tiempos_entrega)``
- Desviación estándar de retrasos: ``np.std(dias_retraso)``
- Máximo y mínimo de reclamos por zona: ``np.max(reclamos_zona)``, ``np.min(reclamos_zona)``

Operaciones vectorizadas (sin bucles)

```
```python
import numpy as np

Sin NumPy (lento con bucles)
tiempos = [1, 2, 3, 4, 5] días de entrega
resultado = []
for t in tiempos:
 resultado.append(t * 1.05) sumar 5% de penalidad

Con NumPy (rápido)
tiempos_arr = np.array([1, 2, 3, 4, 5])
resultado = tiempos_arr * 1.05 operación vectorizada
```
```

Beneficio: procesa 1 millón de entregas en mili-segundos en lugar de segundos.

Operaciones matriciales

- Matriz de correlación entre variables: retrasos vs clima, zona vs demanda
- Transformaciones de escalas para normalización

B. Pandas: Manipulación y exploración de datos tabulares

¿Qué es?

Pandas es una librería para trabajar con datos estructurados en forma de tablas (DataFrames). Está construida encima de NumPy y permite lectura, limpieza, transformación y exploración de archivos CSV y Excel.

¿Para qué sirve en logística?

1. Lectura de archivos Excel y CSV

```
```python
import pandas as pd

Leer archivo Excel de entregas
entregas = pd.read_excel('entregas_diarias.xlsx')

Leer archivo CSV de reclamos
```

```
reclamos = pd.read_csv('reclamos_2026.csv')
```

```
...
```

Automáticamente convierte datos tabulares en DataFrame (tabla estructurada).

## 2. Exploración rápida de datos

```
```python
```

```
# Ver primeras 5 filas
```

```
print(entregas.head())
```

```
# Ver estructura: columnas, tipos, valores nulos
```

```
print(entregas.info())
```

```
# Estadísticas descriptivas
```

```
print(entregas.describe())
```

```
...
```

En segundos se identifica: cuántas entregas hay, qué columnas faltan, qué tipos de datos son.

3. Limpieza y transformación de datos

```
```python
```

```
Identificar valores faltantes
```

```
print(entregas.isnull().sum()) # cuenta nulos por columna
```

```
Rellenar faltantes (imputación)
```

```
entregas['tiempo_entrega'].fillna(entregas['tiempo_entrega'].mean(), inplace=True)
```

```
Renombrar columnas para consistencia
```

```
entregas.rename(columns={'Tiempo Entrega': 'tiempo_entrega'}, inplace=True)
```

```
Convertir tipos de datos
```

```
entregas['fecha_envio'] = pd.to_datetime(entregas['fecha_envio'])
```

```
...
```

## 4. Agrupación y resumen

```
```python
```

```
# Agrupar entregas por zona y calcular promedio de retrasos
```

```
retrasos_por_zona = entregas.groupby('zona')['dias_retraso'].mean()
```

```
# Contar reclamos por tipo
tipos_reclamo = reclamos['tipo_reclamo'].value_counts()
...
```

Responde preguntas clave: ¿Qué zona tiene más retrasos? ¿Cuál es el reclamo más frecuente?

5. Fusión de tablas

```
```python
Combinar tabla de entregas con tabla de reclamos
analisis = pd.merge(entregas, reclamos, on='id_entrega', how='left')
...
```
```

C. Relación con EDA y Clase 4

Según Clase 4 (Análisis Exploratorio de Datos), pandas permite detectar:

- Tendencias: ¿Los retrasos aumentan con el tiempo?
- Outliers: ¿Hay zonas con entregas anormalmente lentas?
- Correlaciones: ¿Clima y retrasos correlacionan?

Pandas es la herramienta que hace posible EDA operativamente.

D. Comparación con Excel manual

| Tarea | Excel manual | Pandas |
|------------------------------------|-------------------------|--|
| Abrir archivo 100 MB | Cuelga o tarda minutos | Carga en segundos |
| Calcular promedio de 1M registros | Manual o lento | Instantáneo |
| Encontrar duplicados | Buscar/Filtro (tedioso) | <code>`df.duplicated().sum()`</code> (1 línea) |
| Rellenar valores faltantes | Manual por celda | <code>`fillna()`</code> automático |
| Cambiar formato de todos los datos | Formateo manual | <code>`to_datetime()`</code> , <code>`astype()`</code> |

Pregunta 03

Por qué son importantes las librerías de visualización

Según Clase 4 (Análisis Exploratorio de Datos), la visualización es la puente entre datos crudos y decisiones empresariales. Sin gráficos, la gerencia no entiende los insights.

A. El rol de la visualización en comunicación

La gerencia de logística NO lee tablas de números; necesita imágenes claras que respondan:

- ¿En qué zonas hay más retrasos?
- ¿Cuál es la tendencia de reclamos mes a mes?
- ¿Qué clientes tienen entregas anormalmente lentas?

Principio: Un gráfico bien hecho comunica en segundos lo que una tabla requiere minutos para entender.

B. Librerías principales

1. Matplotlib

- La librería base para gráficos en Python
- Flexible pero de bajo nivel (requiere muchas líneas de código)
- Uso: histogramas, scatter plots, gráficos de línea

```
```python
import matplotlib.pyplot as plt

Gráfico de retrasos por semana
plt.figure(figsize=(10, 6))
plt.plot(semanas, dias_retrazo, marker='o', label='Días de retraso')
plt.title('Evolución de Retrasos por Semana')
plt.xlabel('Semana')
plt.ylabel('Días de retraso')
plt.legend()
plt.grid()
plt.show()
```
```

2. Seaborn

- Construida encima de Matplotlib, más estética y rápida
- Especializada en estadística e integración con pandas
- Uso: distribuciones, relaciones entre variables, heatmaps

```
```python
import seaborn as sns

Distribución de tiempos de entrega
sns.histplot(entregas['tiempo_entrega'], kde=True)
```

```
plt.title('Distribución de Tiempos de Entrega')
plt.show()
```

```
Boxplot de retrasos por zona (detecta outliers automáticamente)
sns.boxplot(data=entregas, x='zona', y='dias_retraso')
plt.title('Retrasos por Zona (Outliers visibles)')
plt.show()
...
```

### 3. Plotly (alternativa moderna, opcional)

- Gráficos interactivos y dinámicos
- Ideal para dashboards web

## C. Ejemplos prácticos para la empresa logística

### Caso 1: Identificar zonas críticas

```
```python
# Tabla numérica (no clara)
print(entregas.groupby('zona')['dias_retraso'].mean())
Zona A: 1.2
Zona B: 3.8
Zona C: 2.1

# Gráfico de barras (claro)
sns.barplot(data=entregas.groupby('zona')['dias_retraso'].mean().reset_index(),
            x='zona', y='dias_retraso')
plt.title('Retrasos Promedio por Zona')
plt.show()
....
```

La gerencia ve inmediatamente: Zona B es crítica

Caso 2: Tendencia temporal de reclamos

```
```python
Línea temporal de reclamos acumulados
reclamos_diarios = reclamos.groupby('fecha').size()
plt.plot(reclamos_diarios.index, reclamos_diarios.values)
plt.title('Reclamos Diarios - Tendencia')
plt.xlabel('Fecha')
plt.ylabel('Cantidad de reclamos')
plt.show()
...

```



La gerencia ve: ¿aumentan? ¿disminuyen? ¿hay picos?

### Caso 3: Relación entre variables (correlación)

```
```python
# Matriz de correlación visual
corr = entregas[['temperatura', 'dias_retraso', 'reclamos']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlación entre Variables')
plt.show()
```
```

**Pregunta:** ¿clima y retrasos correlacionan?

**Respuesta:** heatmap lo muestra en colores

### D. Importancia estratégica para la empresa

| Beneficio                  | Impacto                                               |
|----------------------------|-------------------------------------------------------|
| Transparencia              | Gerencia ve exactamente en qué falla el proceso       |
| Velocidad de decisión      | No requiere interpretación; el gráfico es evidente    |
| Identificación de patrones | Outliers, tendencias, correlaciones saltan a la vista |
| Reportes ejecutivos        | Presentables directamente a stakeholders              |
| Validación de hipótesis    | ¿La zona B realmente es crítica? El gráfico lo prueba |

La visualización es la última etapa del EDA (Análisis Exploratorio de Datos). Sin gráficos, EDA es solo números; con gráficos, EDA es insight accionable.

### Pregunta 04

Herramienta para escribir y ejecutar código Python de forma ordenada

Existen varias opciones; la elección depende del tipo de proyecto y preferencia del equipo.

### A. Opciones disponibles

1. Jupyter Notebook (Recomendado para análisis y documentación)

#### ¿Qué es?

Entorno interactivo basado en navegador que combina código, texto, imágenes y gráficos en un único documento (`.ipynb`).

**Ventajas:**

- Código ejecutado celda por celda (ideal para exploración interactiva)
- Visualización inmediata de gráficos y tablas
- Documentación integrada (markdown entre celdas de código)
- Excelente para reportes y presentaciones
- Facilita colaboración: compartir notebooks con comentarios

**Desventajas:**

- No ideal para scripts grandes o producción
- Puede ser lento con datasets muy grandes

**Caso de uso en logística:**

Proyecto: "Análisis de Entregas Q1 2026"

notebook: analisis\_entregas\_q1.ipynb

**Contenido:**

[Celda 1: Importar librerías]

[Celda 2: Cargar datos]

[Celda 3: Exploración básica]

[Markdown: "Hallazgo: Zona B tiene 3.8 días retraso promedio"]

[Celda 4: Gráfico de zonas]

[Celda 5: Correlación clima-retraso]

**Instalación:**

pip install jupyter

**2. VS Code + Python Extension (Recomendado para proyectos grandes)****¿Qué es?**

Editor profesional de código con soporte para Python, debugging y ejecución de scripts.

**Ventajas:**

- Ideal para scripts `.py` grandes y organizados
- Integración con Git para control de versiones
- Debugging avanzado (breakpoints, inspección de variables)
- Gestor de extensiones para personalización
- Terminal integrada

## **Caso de uso en logística:**

### **Proyecto: "Pipeline de análisis de entregas"**

#### **Estructura:**

- main.py            script principal
- limpieza.py        funciones de limpieza
- analisis.py        análisis estadístico
- visualizacion.py   generación de gráficos
- data/              archivos CSV/Excel
- resultados/        outputs (gráficos, reports)
- requirements.txt   lista de librerías

#### **Instalación:**

Descargar VS Code desde <https://code.visualstudio.com/>

Instalar extensión "Python" de Microsoft

O usar Python extension pack

## **3. RStudio (Si también usan R)**

### **¿Qué es?**

IDE especializado para R, con paneles para consola, editor, variables y gráficos.

#### **Ventajas:**

- Panel especializado para variables y objetos
- RMarkdown: documentos combinados R + markdown
- Visualización inmediata de plots

#### **Desventajas:**

- Principalmente para R; Python es limitado
- Overkill si solo usa Python

#### **Caso de uso:**

Si la empresa usa TANTO R como Python, RStudio para R y VS Code para Python.

#### **4. Google Colab (Solución temporal sin instalación)**

**¿Qué es?**

**Jupyter Notebook gratuito en la nube (Google Drive).**

**Ventajas:**

- No requiere instalación local
- GPU y TPU gratuita
- Fácil de compartir (link de Drive)

**Desventajas:**

- Requiere internet
- Datos publicitados (privacidad limitada)

**Síntesis: Respuesta integral para la empresa**

**La empresa logística debe seguir este plan de implementación:**

- Verificar hardware (Pregunta 1): i5+, 8GB RAM, SSD 512GB mínimo
- Instalar Anaconda con Python 3.10+ y crear entorno del proyecto
- Usar pandas + NumPy (Pregunta 2) para cargar, limpiar y transformar datos de Excel/CSV
- Visualizar con Seaborn/Matplotlib (Pregunta 3) para comunicar insights a la gerencia
- Escribir código en Jupyter Notebook (Pregunta 4) para análisis colaborativo y documentado

**Este flujo garantiza que el equipo pueda:**

- Importar entregas y reclamos históricos
- Identificar zonas críticas, tendencias y outliers
- Generar reportes visuales para decisiones de negocio

**Referencias**

- Anaconda: <https://docs.anaconda.com/>
- Pandas: <https://pandas.pydata.org/docs/>
- NumPy: <https://numpy.org/doc/>
- Matplotlib/Seaborn: <https://matplotlib.org/>, <https://seaborn.pydata.org/>
- Jupyter: <https://jupyter.org/documentation>